



CSE 3105 / CSE 3137

OBJECT-ORIENTED ANALYSIS AND DESIGN

FALL 2025

COURSE PROJECT: *BagItUp!*

System Design Document

Group 19

<i>Zahra Ismayilli</i>	<i>220315094</i>
<i>Mehmet Mert Yiğit</i>	<i>220315103</i>
<i>Begüm Sezer</i>	<i>220315067</i>
<i>Zeynep Reyryan Kaleli</i>	<i>220315071</i>
<i>Göktuğ Kayacı</i>	<i>230315009</i>
<i>Aykhan Bakhshizada</i>	<i>210315091</i>

30 December 2025

Table of Contents

1	Introduction.....	1
1.1	Purpose of the System	1
1.2	Design goals	2
2	Current Software Architecture	3
3	Proposed Software Architecture	3
3.1	Subsystem decomposition	4
3.1.1	Architectural Pattern	6
3.2	Hardware/software mapping.....	7
3.2.1	Mapping Objects to Hardware	9
3.2.2	Mapping Associations to Connectivity	9
3.2.3	Mapping Constraints and Rationale	10
3.3	Persistent data management.....	10
3.4	Access control and security	11
3.4.1	User model and access matrix.....	11
3.4.2	Key Security Principles.....	16
3.4.3	Authentication mechanism.....	17
3.4.4	Encryption and key management	17
3.4.5	Guarding objects against unauthorized access	18
3.5	Boundary conditions.....	18
3.5.1	System Startup	18
3.5.2	System Shutdown.....	19
3.5.3	Error Management and Recovery	19
3.5.4	Notification and User Alerts	20
3.5.5	Security and Data Privacy	21
4	Subsystem Services.....	22
5	Glossary	23
6	References.....	23
7	Appendix.....	24

Table of Figures

Figure 3.1.1.1 Architectural Diagram with Processes	7
Figure 3.1.1.1 Deployment Diagram of the BagItUp System	8
Figure 3.4.1.1 Access Matrix	11
Figure 3.5.5.1 Boundary Conditions Use Case Diagram.....	21
Figure 3.5.5.1 UML Component Diagram	22

1 Introduction

The BagItUp! system is a food waste reduction platform designed to connect consumers with businesses that have surplus food available for redistribution. The system aims to minimize food waste while promoting sustainable consumption habits through incentive mechanisms such as eco-points and loyalty campaigns. BagItUp! supports both in-store pickup and optional local delivery, addressing limitations observed in existing food rescue platforms.

This System Design Document (SDD) presents the architectural design of BagItUp! based on the requirements identified during the analysis phase. It describes the system's structure, major subsystems, deployment decisions, persistent data management strategy, and security mechanisms. The document provides traceability between system requirements and architectural decisions, ensuring that the proposed design satisfies functional, non-functional, and scalability requirements.

The design of BagItUp! is influenced by existing food waste reduction platforms such as Too Good To Go and Olio; however, it extends these models by supporting modular subsystem decomposition, delivery integration, and sustainability-driven reward systems. The architectural decisions documented here are shaped by constraints related to security, performance, maintainability, and future extensibility.

1.1 Purpose of the System

The primary purpose of the BagItUp! system is to reduce food waste by enabling businesses to offer surplus food items as "Surprise Bags" to consumers at reduced prices. The system acts as a digital marketplace that facilitates secure transactions, order verification, and fair distribution of surplus food.

From a business perspective, BagItUp! provides tools for managing surplus inventory, monitoring sales, running promotional campaigns, and verifying order collection or delivery. From a consumer perspective, the system enables discovery of nearby surplus food, secure pre-payment, transparent order tracking, and reward accumulation through eco-points.

In addition to waste reduction, the system aims to encourage long-term sustainable behavior by integrating a loyalty and eco-point mechanism that rewards environmentally responsible actions. By supporting optional delivery services, BagItUp! also increases accessibility for users who cannot perform in-store pickup, thereby broadening the system's social and environmental impact.

1.2 Design goals

The design of the BagItUp! system is guided by a set of prioritized design goals. These goals influence architectural decisions and reflect trade-offs between performance, security, scalability, and development complexity.

High-Priority Design Goals

- **Scalability:**

The system must support a growing number of users, businesses, and transactions without significant performance degradation. A microservices-based architecture is chosen to allow independent scaling of critical subsystems such as orders, payments, and delivery.

- **Security:**

Since the system handles payments and personal user data, strong security measures are mandatory. This includes secure authentication, role-based access control, encrypted communication, and protection of sensitive data at rest.

- **Reliability and Consistency:**

Order processing, payment handling, and eco-point assignment must be reliable and consistent. The system prioritizes transactional integrity to prevent issues such as duplicate orders, incorrect payments, or lost rewards.

Medium-Priority Design Goals

- **Maintainability:**

The architecture should be modular and easy to maintain. Clear separation between boundary, control, and entity objects enables easier debugging, testing, and future modifications.

- **Extensibility:**

The system should support future enhancements, such as new reward mechanisms, additional delivery options, or integration with external sustainability platforms, with minimal impact on existing components.

Low-Priority Design Goals

- **Client-Side Complexity Reduction:**

The mobile and web clients are kept lightweight by centralizing business logic on the backend. While this may increase server responsibility, it simplifies client maintenance and ensures consistent behavior across platforms.

2 Current Software Architecture

The software architecture of current market-leading food waste reduction platforms, such as Too Good To Go, Olio, and Karma, is predominantly based on a centralized Client–Server Architecture. In these systems, a single backend server layer is responsible for orchestrating all core system services, including business onboarding, product listings, order processing, payment gateway integration, and user data management. Client applications, typically mobile or web-based, act primarily as presentation layers that communicate with the backend through predefined APIs.

These traditional architectures are often implemented using a monolithic backend design, where most business logic is tightly coupled within a single deployable unit. While this approach simplifies initial development and deployment, it relies on several assumptions: that food redistribution workflows are primarily pickup-based, that logistics requirements remain minimal, and that businesses require limited operational customization. For early-stage platforms, this architectural style has proven effective in ensuring system stability and straightforward maintenance.

However, as identified during the BagItUp! requirements analysis, such architectures exhibit notable limitations when extended to more complex operational scenarios. Existing platforms provide restricted support for local delivery logistics, limited adaptability to diverse business models, and insufficient mechanisms for encouraging long-term sustainable behavior through incentive systems. Furthermore, monolithic designs can hinder scalability, as changes or extensions in one functional area may impact the entire system.

To address these challenges, BagItUp! adopts a Modular and Extensible Architecture while preserving the reliability of established client–server principles. Instead of relying on a single monolithic backend, the system is decomposed into specialized subsystems, each responsible for a clearly defined domain of functionality. This architectural approach enhances maintainability, supports independent evolution of subsystems, and allows new features to be integrated with minimal disruption to existing services.

By transitioning to a modular structure, the BagItUp! architecture enables advanced capabilities such as local delivery integration, eco-point reward algorithms, and business-driven campaign management. This design explicitly addresses the shortcomings observed in current market solutions, while providing a scalable foundation capable of supporting future functional and technological growth.

3 Proposed Software Architecture

This section presents the comprehensive system design model for the BagItUp! platform. It bridges the gap between the high-level conceptual requirements and a fully defined technical architecture by systematically addressing the structural, physical, and security-related dimensions of the system. The goal of this section is to provide a clear and organized view of how the platform is constructed, deployed, and protected, ensuring scalability, reliability, and maintainability throughout its lifecycle.

The proposed BagItUp! system adopts a layered, modular, and service-oriented architectural style built upon established client–server principles.

The architecture is organized into presentation, business, and data layers. Presentation-layer components (Mobile Application and Business Dashboard) handle user interaction, business-layer components encapsulate application logic and coordination through independent services, and data-layer components manage persistent system data through a centralized database.

This layered organization ensures separation of concerns, improves maintainability, and enables independent evolution of system subsystems while preserving system-wide consistency and security.

The proposed system architecture is structured into five key subsections, each focusing on a critical aspect of the overall design:

Subsystem Decomposition describes the modular breakdown of the platform into independent yet cooperative subsystems. Each subsystem encapsulates a well-defined domain of responsibility, enabling loose coupling, improved maintainability, and ease of future extensions. This subsection also defines the interactions and dependencies between subsystems and is illustrated using a UML Component Diagram to represent the software architecture.

Hardware/Software Mapping illustrates how the software subsystems are deployed onto physical and virtual hardware nodes. It details the execution environment, server infrastructure, and communication mechanisms, including the protocols used to enable secure and efficient data exchange between components. A UML Deployment Diagram is used to provide a runtime view of the system.

Persistent Data Management explains the platform's data storage strategy, including database selection, schema design principles, and data encapsulation mechanisms. Emphasis is placed on ensuring transactional consistency, data integrity, and secure long-term storage while supporting scalable access patterns.

Access Control and Security defines the system's security architecture by outlining authentication mechanisms, role-based authorization policies, encryption techniques, and key management strategies.

Boundary Conditions outline the system's behavior during critical operational phases such as start-up, shutdown, error handling, and recovery scenarios and are illustrated using a UML Use Case Diagram.

3.1 Subsystem decomposition

The BagItUp! system includes different subsystems. These subsystems are responsible for different aspects of the application and its operation:

Mobile Application (Consumer)

This subsystem provides the user interface for consumers to interact with the platform. It allows consumers to browse and purchase available "Surprise Bags."

Bag Discovery: Displays available Surprise Bags based on location.

Payment System: Handles secure transactions.

QR Code Generation: Generates unique codes for collection verification.

Eco-Point System: Tracks and awards eco-points for purchases.

Order Management: Manages consumer orders and allows tracking.

Business Dashboard (Business)

The Business Dashboard is a web or mobile-based interface that allows businesses to manage their food surplus listings.

Listing Management: Businesses can add, update, or remove Surprise Bags.

Sales Management: Displays ongoing sales data and available quantities.

Collection Verification: Verifies consumer pickups by scanning QR codes.

Promotion and Campaign Management: Businesses can create and manage promotional campaigns.

Delivery Management: Supports delivery options for businesses.

Payment Gateway Subsystem

Handles all financial transactions securely and ensures payment processing is reliable and safe.

Payment Processing: Supports to secure pre-payment transactions.

Transaction Validation: Ensures accurate processing of payments.

Data Security: Complies with industry-standard security protocols.

Order Management Subsystem

Controls the cycle of each order, including creation, verification and status tracking,

Order: Tracks the creation of orders after payment.

Order Tracking: Updates order status throughout the process.

Verification Code Generation: Generates QR codes to verify collection.

Delivery Subsystem

Supports delivery when consumers opt for home delivery instead of in-store pickup.

Courier Assignment: Assigns a courier to handle the delivery of the order.

Delivery Tracking: Tracks the status of the delivery in real-time.

Delivery Confirmation: Verifies delivery via QR code scanning.

Eco-Points & Loyalty System

Manages the eco-points and loyalty rewards system to encourage sustainable behavior.

Eco-Point Accumulation: Awards points to consumers for each purchase.

Coupon Generation: Allows consumers to redeem points for discounts.

Campaigns and Rewards: Enables businesses to run loyalty campaigns

3.1.1 Architectural Pattern

The BagItUp! system utilizes a Microservices Architecture to ensure scalability, flexibility, and maintainability. Each subsystem can function independently, making the system easy to update and scale. Microservices are small, independent services that focus on specific tasks and communicate through APIs. This architecture allows the system to grow and evolve over time.

3.1.1.1 Microservices Components

Mobile Application: Communicates with the backend to fetch bag listings, place orders, process payments, and generate QR codes.

Business Dashboard: A microservice that allows businesses to manage food surplus listings and verify collection.

Payment Service: Manages the payment processing and transaction validation.

Order Management Service: Handles order creation, updates, and status tracking.

Delivery Service: Manages courier assignments and delivery tracking for orders with delivery.

Eco-Points Service: Tracks and manages the eco-points system and rewards.

3.1.1.2 Communication Between Microservices

Consumer and Business Services: The Consumer app requests Surprise Bag data from the Business service.

Payment and Order Services: The Payment service triggers the creation of an order in the Order service.

Order and Delivery Services: If delivery is selected, the Order service communicates with the Delivery service to handle courier assignments.

Order and Eco-Points Services: After an order is completed, the Order service triggers the Eco-Points service to award points.

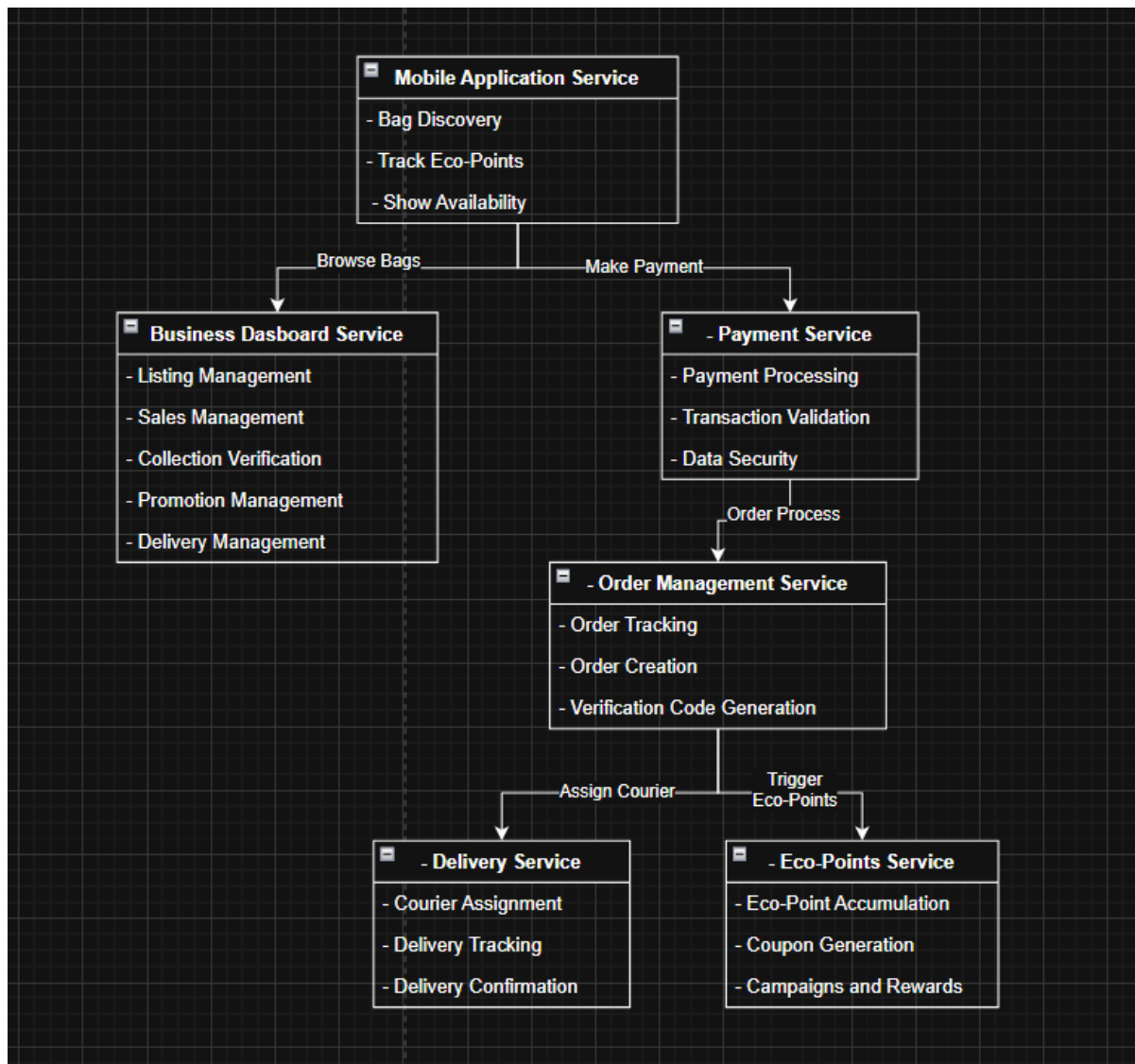


Figure 3.1.1.1 Architectural Diagram with Processes

3.2 Hardware/software mapping

Hardware software mapping describes how the software elements of the BagItUp system are allocated to physical hardware resources at run time and how these elements communicate with each other. This activity connects the logical design of the system to its physical realization by deciding where computation takes place, where data is stored, and how users and external systems interact with the software. In accordance with the course approach, control objects are mapped to processors where computation and coordination occur, entity objects are mapped to memory and

persistent storage where long-term data is managed, and boundary objects are mapped to input/output devices close to users and external interfaces.

The overall allocation of BagItUp subsystems to physical devices and their communication relationships is illustrated using a UML Deployment Diagram. This diagram provides a runtime view of the system, showing physical nodes, deployed subsystems, and communication links between them.

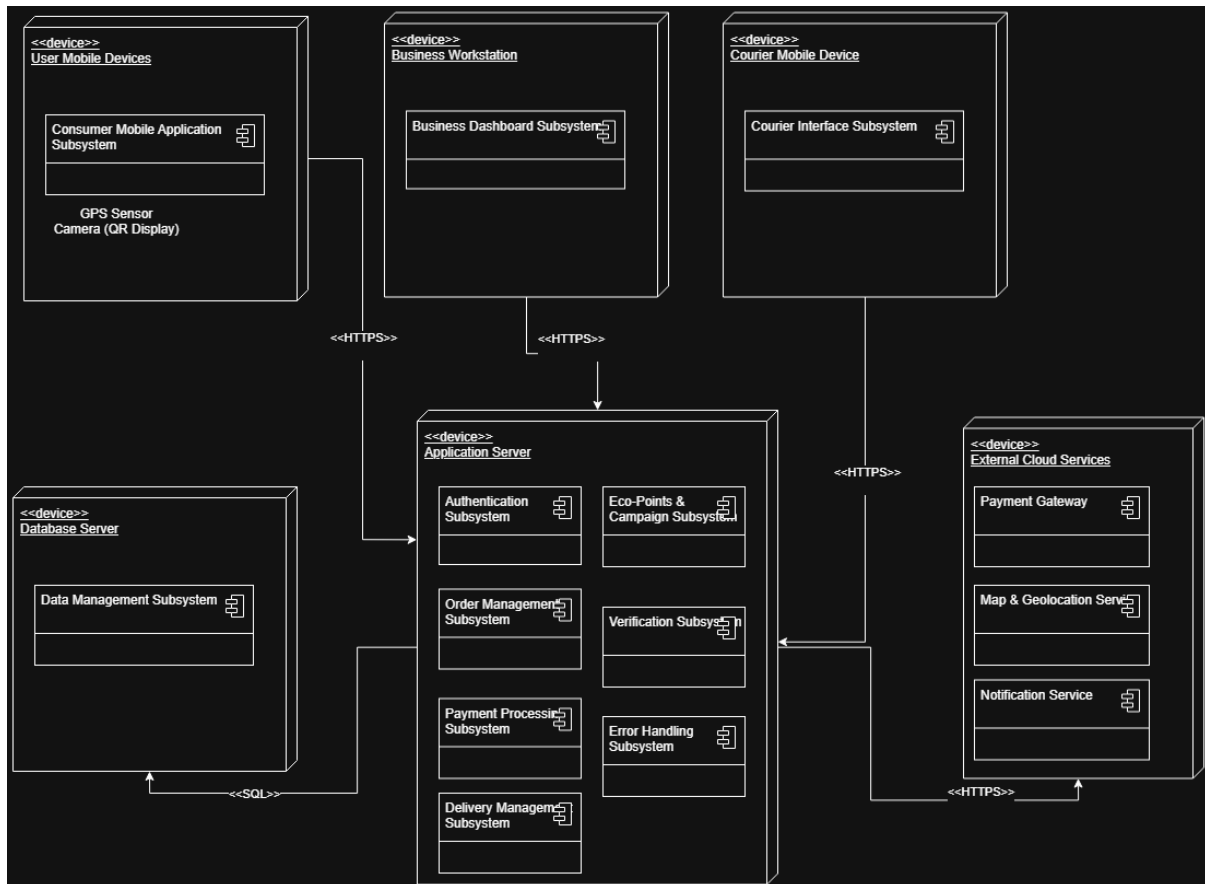


Figure 3.1.1.1 Deployment Diagram of the BagItUp System

- User Mobile Devices**
 Host the Consumer Mobile Application Subsystem and handle user interaction, location access, and QR code display. These devices do not execute business logic or access the database directly.
- Business Workstation**
 Hosts the Business Dashboard Subsystem, allowing businesses to list Surprise Bags, manage campaigns, and verify QR codes. All operations are performed through the Application Server.
- Courier Mobile Device**
 Hosts the Courier Interface Subsystem used for delivery assignment, tracking, and QR code verification during delivery. Business logic is handled by the backend.

- **Application Server**
Hosts all core control subsystems, including authentication, order management, payment processing, delivery management, eco-points, and verification logic. It coordinates all system workflows.
- **Database Server**
Hosts the Data Management Subsystem and stores all persistent system data such as users, orders, payments, and eco-points. It is accessed only by the Application Server.
- **External Cloud Services**
Provide external functionality such as payment processing, map and geolocation services, and notification delivery. These services are accessed via secure network connections.

3.2.1 Mapping Objects to Hardware

BagItUp follows a client server architecture. User interaction is handled on client devices, core application logic is centralized on the backend server, and persistent data is managed by a database server.

- **Boundary objects to input/output devices:**
Boundary objects such as the MobileApp (consumer interface), BusinessDashboard (business interface), Payment Gateway UI, and Delivery Assignment Interface are deployed on user-facing devices including mobile phones, tablets, and personal computers. These components rely on input/output resources such as touch screens, cameras for QR code display and scanning, GPS sensors for location-based discovery, and network interfaces for communication with backend services.
- **Control objects to processor:**
Control objects including OrderController, PaymentController, CouponController, DeliveryController, and AuthenticationController are deployed on the backend application server. This server provides sufficient processing power to handle request coordination, payment validation, order workflows, delivery management, and concurrent access during peak usage periods.
- **Entity objects to memory and persistent storage.**
Entity objects such as User, Consumer, Business, Courier, SurpriseBag, Order, Payment, EcoPoints, Coupon, PromotionCampaign, and VerificationCode are stored in the database server. These objects represent long-lived system data and are persisted to ensure reliability and consistency. Frequently accessed data may be temporarily cached in server memory to meet performance requirements, such as fast loading of nearby Surprise Bags on the map.

3.2.2 Mapping Associations to Connectivity

Associations between objects that span different subsystems are realized as network connections. The mapping distinguishes between physical connectivity, which defines concrete communication links and protocols, and logical connectivity, which specifies how subsystems interact at the design level.

- **Physical connectivity.**
Client devices communicate with the application server using secure HTTPS connections for operations such as browsing Surprise Bags, creating orders, initiating payments, and

verifying QR codes.

The application server communicates with the database server through SQL-based connections to read and write User, SurpriseBag, Order, and EcoPoints data.

Communication between the application server and storage services uses HTTPS to store and retrieve QR codes, digital receipts, and system logs.

External cloud services such as payment providers, map and geolocation services, and notification systems are accessed via HTTPS-based APIs.

- **Logical connectivity.**

Client-side subsystems, including the MobileApp and BusinessDashboard, never access the database directly. All data access is mediated through backend services. This design enforces security, validation, and consistency, and ensures a single source of truth for all system data.

3.2.3 Mapping Constraints and Rationale

The hardware and software mapping of BagItUp is shaped by design goals and external constraints. Payment processing is handled by an existing payment gateway to meet security requirements and reduce risk. Location-based features rely on external map and geocoding services. To ensure high availability, backend services are kept stateless so multiple application servers can run in parallel behind a load balancer. Database and storage services are managed and include backup and recovery support. Centralizing all business logic on the server allows the mobile application to work consistently on both iOS and Android without duplicating core functionality on client devices.

3.3 Persistent data management

The persistent data management layer in the BagItUp! system is designed to provide high reliability, scalability, and security for all long-term system data. Given the complex and interrelated nature of system actor including consumers, food providers, couriers, and administrators the system requires a robust data storage solution capable of handling transactional consistency and concurrent access.

To meet these requirements, BagItUp! utilizes a centralized Relational Database Management System (RDBMS). This choice ensures the enforcement of ACID properties, which are essential for maintaining data integrity, particularly during critical operations such as payment processing, order confirmation, and inventory updates during peak usage periods. Relational modeling also supports structured relationships between users, orders, surprise bags, deliveries, and eco-point records.

The persistence strategy is based on three fundamental pillars:

Data Modeling

The system persistently stores a diverse set of data entities, including encrypted user profiles, authentication credentials, and role-based access information. It also maintains real-time surprise bag inventory levels, order and payment transaction histories, delivery records, and historical eco-point accumulation data. These entities are structured using normalized database schemas to minimize redundancy and ensure consistency.

Encapsulation

To ensure separation of concerns and protect sensitive data, direct database access is strictly prohibited from boundary components such as the Mobile Application and Business Dashboard. All data interactions are encapsulated within backend services using the Data Access Object (DAO) pattern. This abstraction layer centralizes database logic, simplifies maintenance, and allows the underlying database technology to be modified without impacting higher-level system components.

Security at Rest

Security is a core consideration in persistent data management. Sensitive information, including user credentials, verification tokens, and payment-related references, is protected using managed encryption mechanisms. This approach minimizes the risk of unauthorized data exposure and aligns with best practices for secure data storage in distributed systems.

3.4 Access control and security

This section defines who can access what in BagItUp using an access matrix (actors as rows, classes/resources as columns) and explains the main security decisions: authentication, encryption, and key management.

3.4.1 User model and access matrix

Actor / Class	User	SurpriseBag	Order	Payment	VerificationCode	EcoPoints	Coupon / Campaign	Admin Panel
Consumer	<<read>> viewProfile() <<update>> updateProfile()	<<read>> viewBags() searchByLocation()	<<create>> createOrder() <<read>> viewOwnOrders()	<<create>> initiatePayment() <<read>> viewStatus()	<<read>> viewQR()	<<read>> viewBalance()	<<read>> viewCoupons() redeemCoupon()	—
Business (Employee)	<<read>> viewBusinessProfile() <<update>> updateIntro()	<<create>> createBag() <<update>> updateBag() <<read>> viewOwnBags()	<<read>> viewOrders() <<update>> confirmPickup()	<<read>> viewPaymentStatus()	<<read>> scanQR() <<update>> verifyCode()	<<read>> viewStats()	<<create>> createCampaign() <<update>> updateCampaign()	—
Courier	<<read>> viewProfile()	<<read>> viewAssignedBag()	<<read>> viewAssignedOrders() <<update>> markDelivered()	<<read>> viewPaymentStatus()	<<read>> scanQR() <<update>> confirmDelivery()	<<read>> viewStats()	—	—
System Admin	<<read>> viewUsers() <<update>> suspendUser()	<<read>> viewAllBags()	<<read>> viewAllOrders() <<update>> resolveDispute()	<<read>> viewAllPayments() <<update>> refundPayment()	<<read>> auditCodes()	<<read>> viewAllPoints()	<<read>> moderateCampaigns() <<delete>> disableCampaign()	<<fullaccess>>

Figure 3.4.1.1 Access Matrix

BagItUp has different user roles (actors) that need different rights:

1. Consumer

User

- «read» – View own profile information
- «update» – Modify personal details
- viewProfile() – Display profile data
- updateProfile() – Update personal information

SurpriseBag

- «read» – Browse available Surprise Bags
- viewBags() – View bag listings
- searchByLocation() – Search bags by location

Order

- «create» – Place a new order
- «read» – View own orders only
- createOrder() – Create a purchase order
- viewOwnOrders() – Display order history

Payment

- «create» – Initiate payment for an order
- «read» – View payment status
- initiatePayment() – Start payment process
- viewStatus() – Check payment result

VerificationCode

- «read» – View QR code for pickup
- viewQR() – Display verification QR

EcoPoints

- «read» – View earned eco-points

- viewBalance() – Check eco-point balance

Coupon / Campaign

- «read» – View available coupons
- redeemCoupon() – Redeem coupon during purchase

Admin Panel

NO ACCESS

2. Business (Employee)

User

- «read» – View business profile
- «update» – Update business information
- viewBusinessProfile() – Display business details
- updateInfo() – Modify profile data

SurpriseBag

- «create» – Create new Surprise Bags
- «update» – Update bag details
- «read» – View own bags
- createBag() – Add bag listing
- updateBag() – Modify bag information
- viewOwnBags() – View published bags

Order

- «read» – View orders related to the business
- «update» – Confirm pickup
- viewOrders() – List active orders
- confirmPickup() – Confirm QR-based pickup

Payment

- «read» – View payment status
- viewPaymentStatus() – Monitor transactions

VerificationCode

- «read» – Scan QR code
- «update» – Verify pickup
- scanQR() – Scan code
- verifyCode() – Validate order pickup

EcoPoints

- «read» – View statistics
- viewStats() – Monitor eco-point performance

Coupon / Campaign

- «create» – Create promotional campaigns
- «update» – Update campaigns
- createCampaign() – Launch promotion
- updateCampaign() – Modify promotion

Admin Panel

NO ACCESS

3. Courier

User

- «read» – View courier profile
- viewProfile() – Display profile data

SurpriseBag

- «read» – View assigned Surprise Bags
- viewAssignedBag() – Access bag details

Order

- «read» – View assigned orders
- «update» – Confirm delivery
- viewAssignedOrders() – List deliveries

- markDelivered() – Mark order as delivered

Payment

- «read» – View payment status
- viewPaymentStatus() – Check transaction result

VerificationCode

- «read» – Scan QR code
- «update» – Confirm delivery
- scanQR() – Scan delivery QR
- confirmDelivery() – Validate delivery

EcoPoints

- «read» – View delivery statistics
- viewStats() – Monitor performance

Coupon / Campaign

NO ACCESS

Admin Panel

NO ACCESS

4. System Admin

User

- «read» – View user accounts
- «update» – Suspend or manage users
- viewUsers() – List users
- suspendUser() – Disable accounts

SurpriseBag

- «read» – View all Surprise Bags
- viewAllBags() – Inspect listings

Order

- «read» – View all orders
- «update» – Resolve disputes
- viewAllOrders() – Monitor system activity
- resolveDispute() – Handle conflicts

Payment

- «read» – View all payments
- «update» – Refund transactions
- viewAllPayments() – Monitor payments
- refundPayment() – Process refunds

VerificationCode

- «read» – Audit verification codes
- auditCodes() – Review QR usage

EcoPoints

- «read» – View all eco-point data
- viewAllPoints() – System-wide statistics

Coupon / Campaign

- «read» – Moderate campaigns
- «delete» – Disable campaigns
- moderateCampaigns() – Review promotions
- disableCampaign() – Deactivate misuse

Admin Panel

- «full access» – Complete administrative control

3.4.2 Key Security Principles

1. Principle of Least Privilege

Consumers access only their own data

Business users manage only their own listings and orders

Couriers access assigned deliveries only

System Admin has global access limited to maintenance tasks

2. Separation of Duties

Business operations (Business role)

Delivery handling (Courier role)

System oversight and refunds (System Admin)

3. Domain Isolation

Payment data isolated from business operations

User data protected from other roles

Administrative actions restricted by role

3.4.3 Authentication mechanism

BagItUp requires authentication because it is a multi-user system and includes payments and user-specific data.

- **Primary method:** username or email plus password (standard login UI in MobileApp and BusinessDashboard).
- **Session handling:** after successful login, the backend issues a short-lived session token (for example JWT or server session id). The client sends it with every request.
- **Role-based authorization:** every request is checked for role (Consumer vs Business vs Courier vs Admin) before executing the operation.

This directly supports the security requirement that pre-payment must comply with industry-standard security protocols.

3.4.4 Encryption and key management

- **In transit:** all communication between clients and backend uses HTTPS (TLS) to protect credentials, payment initiation, and QR verification traffic.
- **At rest:** sensitive stored data (user credentials, tokens, verification codes, receipts) is protected in the database and storage using managed encryption (cloud provider encryption at rest).

- **Passwords:** never stored as plain text; store salted, hashed passwords.
- **Keys and secrets:** API keys for payment, maps, and notifications are stored in a secure secrets manager (not inside the app). Only the backend can read these secrets.

3.4.5 Guarding objects against unauthorized access

BagItUp enforces access control in the backend using a “check first, then do” approach:

- AuthenticationController validates credentials and issues tokens.
- For each operation, a backend guard checks:
 1. user is authenticated
 2. user role is allowed (from the access matrix)
 3. user owns the resource when required (example: Consumer can only view own orders)
- If any check fails, the request is rejected (access denied) and logged.

3.5 Boundary conditions

3.5.1 System Startup

Ensures that the BagItUp! System initializes properly and functions smoothly for both Consumers and Businesses upon launching the app.

Automatic:

1. **Database Initialization:** On startup, the system checks connectivity to the database and ensures all user data (Business listings, Consumer accounts, eco-points, and transaction records) are up-to-date and accessible.
2. **Payment Gateway Connection:** The system verifies that the payment gateway is active and functional, allowing secure transactions for all users.
3. **Location Services:** The system initializes the geolocation services to provide accurate nearby “Surprise Bag” listings for Consumers.
4. **Push Notifications:** Push notifications are activated to keep Consumers and Businesses informed about updates, such as new Surprise Bags, order statuses, and promotional offers.

Manual Controls by Business Manager:

1. **System Health Check:** The Business Manager can manually trigger a health check to ensure that the server, database connections, and payment systems are operational.
2. **Service Monitoring:** The Business Manager can monitor key system performance indicators to ensure the application is working without errors,

3.5.2 System Shutdown

Safely shut down the BagItUp! System, ensuring that no active transactions are disrupted and all user data is securely saved.

Automatic:

1. **Data Backup:** Before shutting down, the system will back up all transaction logs, user data, and eco-point records to ensure no loss of information.
2. **Payment Finalization:** The system will ensure that any pending payments or orders are either completed or refunded.
3. **Session Termination:** All active user sessions (Consumers, Businesses, and Couriers) will be closed, and users will be logged out gracefully to prevent any loss of information or incomplete orders.

Manual Controls by Business Manager:

1. **Shutdown:** The Business Manager can initiate a manual shutdown, ensuring that data is backed up, and all active sessions are logged off.
2. **User Notification:** The Business Manager will notify users of the planned shutdown via push notifications or in-app alerts, reducing user frustration.

The system should shut down ensuring all data is safely stored, and all user sessions are properly closed.

3.5.3 Error Management and Recovery

Automatically recovering from common errors and ensure minimal disruption to users. Critical issues can be escalated for Business Manager intervention.

Automatic:

1. **Payment Failures:** If a payment fails due to a network issue, the system will automatically retry the payment up to 3 times. After 3 failed attempts, the system will notify the user and initiate a refund.
2. **Database Connectivity Issues:** The system will attempt to reconnect to the If unsuccessful, an alert will be sent to the Business Manager.
3. **Order Timeout:** If a Consumer does not confirm pickup or delivery within the specified window, the system will automatically mark the order as expired and issue a full refund.
4. **Listing Failures:** If a Business tries to list a Surprise Bag but the action fails (e.g., due to network issues), the system will save the listing data locally and retry the listing once the connection is restored.

Manual Controls by Business Manager:

1. **Error Logs & Recovery:** The Business Manager has access to detailed logs and can manually initiate error recovery processes for critical issues, like payment gateway failure or database connection issue.
2. **Error Notifications:** The Business Manager can configure real-time alerts for any critical failures, such as failed transactions or database connectivity issues.

The system should recover from minor issues automatically, while the Business Manager is notified of major problems for prompt resolution. User disruptions should be minimized.

3.5.4 Notification and User Alerts

Keeping all system users informed about system statuses, transaction updates, and errors is this systems priority.

Automatic Notification:

1. **Order Updates:** Consumers and Businesses will receive real-time push notifications about order statuses, such as successful payments, collection confirmations, or delivery updates.
2. **Error Alerts:** Users will be notified in case of issues, such as payment failures or unlisted Surprise Bags.
3. **Scheduled Maintenance:** Users will be notified ahead of time about any planned maintenance or downtimes.

Manual Controls by Business Manager:

1. **Custom Notifications:** The Business Manager can create custom notifications for specific events or alerts, such as low stock warnings for Businesses or system maintenance alerts for all users.

3.5.5 Security and Data Privacy

For protecting the integrity and privacy of user data, including personal and payment information.

1. **Encryption:** All communication, especially payment transactions, will be encrypted using industry-standard protocols.
2. **User Authentication:** Multi-factor authentication will be enforced for all business and admin accounts to ensure secure access.
3. **Data Backup:** All user and transaction data will be backed up securely at regular intervals to prevent loss.

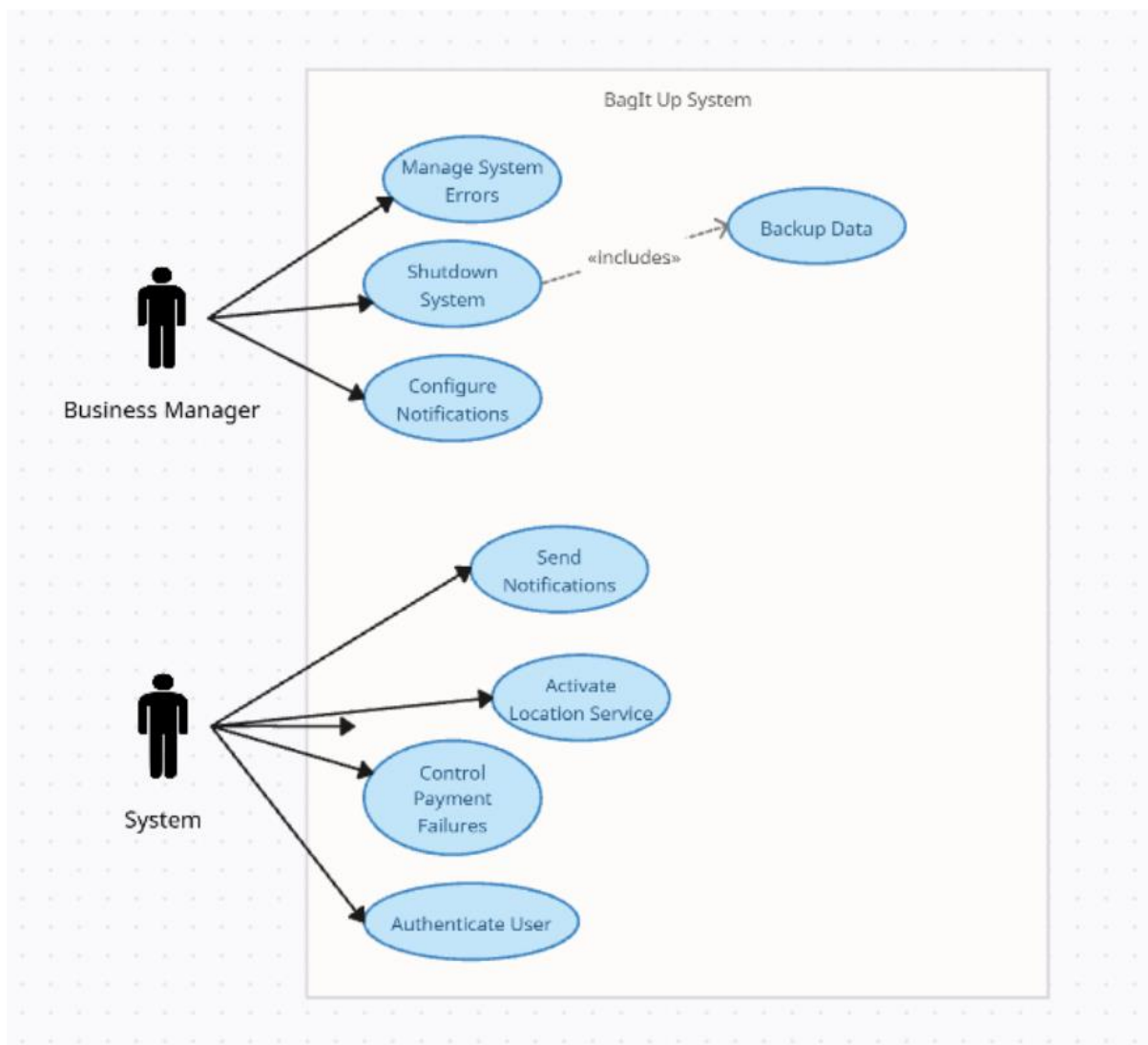


Figure 3.5.5.1 Boundary Conditions Use Case Diagram

4 Subsystem Services

The BagItUp! system is decomposed into a set of specialized subsystems, each responsible for a distinct functional domain. These subsystems expose their functionality through well-defined service interfaces, ensuring loose coupling and clear responsibility boundaries. The provided services and their interactions are illustrated in the UML Component Diagram using lollipop notation.

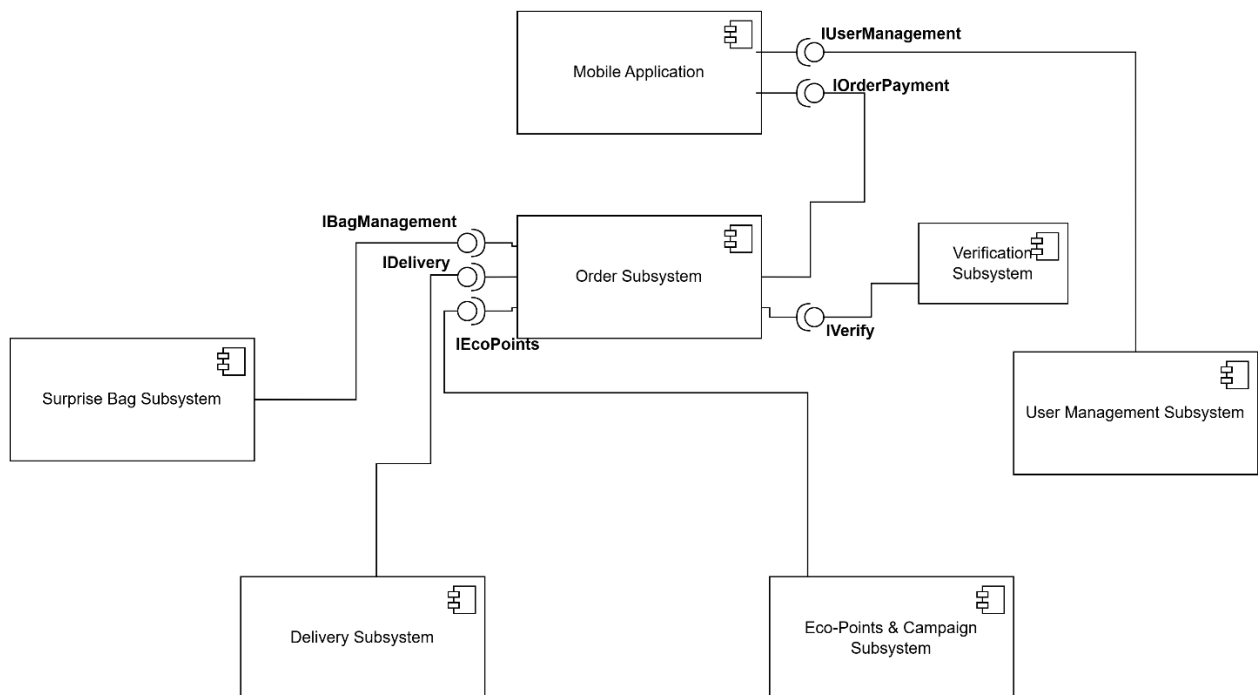


Figure 3.5.5.1 UML Component Diagram

User Management Subsystem (IUserManagement)

The User Management Subsystem is responsible for managing the complete user lifecycle. It provides services for user authentication and authorization, including login(), logout(), and registerUser(). Additionally, it offers manageUserProfile() functionality, enabling the system to maintain accurate user preferences and location data, which are critical for personalized discovery and recommendation features.

Surprise Bag Subsystem (IBagManagement)

This subsystem manages the digital marketplace assets of the platform. It enables businesses to create and update surprise bag offerings through services such as createBag() and updateBag().

Consumers interact with this subsystem via `viewBags()`, which supports location-based filtering and real-time availability, ensuring accurate and timely access to surplus food offerings.

Order Subsystem (IOrderPayment)

Acting as the central transaction engine, the Order Subsystem coordinates the entire order lifecycle. It provides core services such as `createOrder()` and `initiatePayment()`, ensuring secure and consistent checkout operations. This subsystem interacts with multiple other subsystems to validate users, confirm inventory availability, and finalize transactions.

Verification Subsystem (IVerify)

The Verification Subsystem ensures secure and trustworthy order fulfillment. It provides `generateQR()` services for consumers upon successful order placement and `verifyCode()` services for businesses during pickup or delivery confirmation. This mechanism prevents fraud and guarantees that orders are completed only by authorized parties.

Delivery Subsystem (IDelivery)

The Delivery Subsystem supports optional logistics operations for local redistribution scenarios. It provides services such as `assignCourier()`, `trackDeliveryStatus()`, and `confirmDelivery()`, enabling real-time delivery coordination and transparent order tracking for users and businesses.

Eco-Points & Campaign Subsystem (IEcoPoints)

This subsystem manages sustainability-driven incentives and marketing features. It provides `awardEcoPoints()` services to reward users for completed orders and sustainable behavior. Additionally, it allows businesses to design promotional strategies through `createCampaign()`, supporting engagement and long-term user retention.

Together, these subsystems form a cohesive yet flexible architecture that supports the functional, scalability, and sustainability goals of the BagItUp! platform.

5 Glossary

This section provides definitions for the key terms, acronyms, and abbreviations used throughout this System Design Document.

Actor

An external entity that interacts with the BagItUp! system, such as Consumer, Business, Courier, or System Admin.

Boundary Object

A system component responsible for handling interaction between the system and external actors or systems, such as user interfaces and external service gateways.

Client–Server Architecture

An architectural model in which client applications request services from centralized backend servers that manage application logic and data.

Control Object

An object that coordinates system operations, manages workflows, and controls the execution of use cases.

Eco-Points

A reward mechanism that grants sustainability points to consumers based on completed purchases, encouraging environmentally responsible behavior.

Entity Object

An object that represents persistent data stored in the system, such as User, Order, SurpriseBag, Payment, or EcoPoints.

Microservices Architecture

An architectural approach where the system is composed of independent, loosely coupled services that communicate via well-defined interfaces.

QR Code

A machine-readable code generated by the system and used to verify order pickup or delivery.

RDBMS

Relational Database Management System; a database system that stores structured data in relational tables and enforces consistency constraints.

Subsystem

A modular part of the system responsible for a specific set of related functionalities.

UML

Unified Modeling Language; a standardized modeling language used to describe system structure and behavior.

6 References

7 Appendix

- Annex – I: Distribution of Work
- Annex – II: Meeting Minutes

Distribution of Work

This section describes the distribution of work among the group members for the **System Design Document** phase of the BagItUp! project.

All contributions were planned and executed collaboratively, and the workload was distributed in a balanced manner. Each member was responsible for specific sections of the document while actively participating in group discussions, design decisions, and reviews.

- **Aykhan Bakhshizada** was responsible for the Introduction section, including *Purpose of the System* and *Design Goals*. He contributed to defining the overall system vision, objectives, and design priorities.
- **Mehmet Mert Yiğit** worked on the Current Software Architecture and Proposed Software Architecture sections. His contribution focused on architectural analysis, comparison with existing systems, and defining the high-level architectural structure of BagItUp!.
- **Begüm Sezer** was responsible for Access Control and Security and Hardware/Software Mapping. She designed the security model, access matrix, and deployment-related decisions, ensuring system security and proper allocation of software components to hardware resources.
- **Gökтуğ Kayacı** contributed to Subsystem Decomposition and Boundary Conditions. He focused on modular system structure, subsystem responsibilities, and system behavior during startup, shutdown, and error conditions.
- **Zahra Ismayilli** was responsible for Persistent Data Management and Subsystem Services. Her work included defining the data persistence strategy, database design principles, and subsystem service interfaces.
- **Zeynep Reyhan Kaleli** prepared the Glossary, References, and Annex – I / II (Meeting Minutes) sections. She ensured terminology consistency, proper citation of sources, and accurate documentation of project meetings.

All team members participated equally in meetings, design reviews, and final document validation, ensuring coherence and consistency across all sections.

Meeting Minutes

Date:	19.12.2025
Location:	University Campus
Duration:	1 hour
Participants:	All members
Content of the meeting	
The first meeting focused on understanding the project requirements and expectations of the OOAD course. The assignment description and grading criteria were reviewed in detail. The group discussed the overall idea of the BagItUp! system, its purpose, and its sustainability-oriented goals. Key system actors were identified, and the general system scope was agreed upon. An initial discussion was held on possible architectural approaches and similar existing platforms. Preliminary responsibilities were distributed, and members agreed on individual research tasks to be completed before the next meeting.	

Meeting Minutes

Date:	22.12.2025
Location:	University Campus
Duration:	1.5 hour
Participants:	All members
Content of the meeting	
This meeting concentrated on system design decisions. The group finalized the high-level software architecture and agreed on using a Microservices Architecture combined with Client–Server principles. Subsystem decomposition was discussed, and responsibilities of major subsystems were clarified. The access control model and main user roles (Consumer, Business, Courier, System Admin) were reviewed. Based on these decisions, detailed work distribution for document sections and UML diagrams was finalized. Each member confirmed their assigned sections and deadlines.	

Meeting Minutes

Date:	25.12.2025
Location:	online
Duration:	45 minutes
Participants:	All members
Content of the meeting	
The third meeting was dedicated to reviewing and refining the drafted System Design Document. Completed sections were evaluated for consistency, clarity, and alignment with OOAD principles. Particular attention was given to security, persistent data management, hardware/software mapping, and boundary conditions. Necessary revisions and improvements were identified, and missing elements were assigned to relevant team members. The group also agreed on the structure of the Glossary, References, and Annex sections. Final coordination decisions were made to ensure a coherent and well-integrated document before submission.	

